

LuminaX Round 1 research report

Guarded-history safe-cardinality matching under incomplete and delayed information

Item	Submission detail
Team	LuminaX
Members	Ahan Sardar and Enakshee Mondal
Repository	https://github.com/ahansardar/sequential-matching-hackathon
Selected policy	Guarded-history safe-cardinality
Policy version	guarded-history-2.1
Prepared	9 October 2026

Executive summary

We model the task as sequential matching on a changing graph. Every feasible edge is a pair that satisfies both people's hard requirements. Every daily batch is a matching, so no person appears twice. The policy first protects feasibility, then ranks feasible edges, then decides how to allocate the full pool.

Our selected method makes four decisions:

1. It asks up to four available members for missing hard constraints.
2. It removes every pair that is not reciprocally feasible.
3. It ranks feasible pairs by observed soft compatibility. From day 20, a bounded history term breaks close choices.
4. It uses a global maximum-cardinality batch only when that batch contains more pairs and does not lower the total policy score.

The strongest evidence is a fresh 120-episode matched holdout. Guarded history scored 0.2708 MSMI per 100 arrived members, compared with 0.2500 for the same policy without history. That is five additional MSMI outcomes across the block. The paired 95% bootstrap interval for the score difference was $[-0.0167, 0.0583]$, so the measured gain is promising but not statistically clear. We state that uncertainty directly.

The global allocation branch is real but selective. On 1,080 audited decision days, it replaced the greedy batch 26 times. This was 4.45% of days with at least one feasible edge. More aggressive global matching and several learned or hand-designed scorers failed fresh-seed tests, so they are not in the submission runtime.

The selected policy is deterministic, offline, and dependency-free at inference. It uses only the observable request. It never uses hidden simulator truth, private seeds, member IDs as predictors, future feedback, or network access.

1. Problem interpretation

This is not independent pair ranking. It is a sequential decision problem with five linked difficulties:

- Eligibility is reciprocal. One person's interest cannot override the other person's constraint.
- Information is incomplete. Unknown hard fields can block otherwise plausible pairs.

- The pool changes. Members arrive, become unavailable, return, or remain unmatched.
- Outcomes arrive late. A policy must not treat pending feedback as a negative label.
- Pair choices compete. Selecting one edge removes both members from the rest of that day's graph.

The last point matters. The highest-scoring individual edge can block two other good edges. A policy must therefore decide both which pairs look promising and which set of pairs uses the pool well.

1.1 Primary objective

The official primary metric is Mutual Second-Meeting Intention, or MSMI, per 100 arrived members. A pair counts once only when:

1. both people respond to the introduction;
2. both answer Yes;
3. a date happens within 30 days of assignment; and
4. both request a second meeting within three days of the date.

For one episode with N arrived members:

$$\text{episode MSMI score} = 100 * \text{successful MSMI pairs} / N$$

One successful pair among 200 members scores 0.5. The competition score is the equally weighted mean of the six scenario-family means. The score is not a probability and is not capped at 1.

The official tie-break order is:

1. distinct-member introduction coverage;
2. mutual acceptances per 100 arrived members;
3. lower clarification cost; and
4. lower inference time.

We use this order when comparing policies. We do not trade primary score for a better secondary metric.

1.2 Decision objective

The policy follows a strict hierarchy:

```

validity and reciprocal feasibility
      ↓
primary MSMI score
      ↓
official tie-break order

```

A high compatibility score never repairs an infeasible edge. A member may remain unmatched when no feasible edge exists.

2. Research questions and hypothesis

We ask four questions:

1. Can a global allocator serve more members without accepting a lower-scoring batch?
2. Can mature response history improve close choices without overpowering observed compatibility?
3. Can clarification unlock hard-feasible edges without wasting the daily budget?
4. Do more complex pair models repeat their apparent gains on untouched seeds?

Our main hypothesis is:

A constraint-first policy that combines observed compatibility, mature response history, and guarded maximum-cardinality allocation can improve MSML over greedy matching without risking reciprocal feasibility.

The hypothesis has three testable design claims:

- Hard constraints must remain separate from pair scoring.
- History should affect decisions only after feedback has had time to arrive.
- Global allocation should replace greedy allocation only when it adds pairs without lowering the batch's total policy score.

3. Observable information and leakage controls

The policy uses only fields present in the JSON request:

- arrived members and current availability;
- observed questionnaire fields and each field's status;
- past introductions;
- feedback whose `observed_day` has arrived;
- the current day and remaining ask budget; and
- a small policy-owned JSON memory object.

The policy does not use:

- hidden member truth or latent response probabilities;
- the simulator world seed or private evaluation seeds;
- future arrivals or future feedback;
- member IDs as features;
- organiser files or reconstructed generator state; or
- state carried across independent episodes.

Historical logging propensities are null. We therefore do not claim unbiased inverse-propensity evaluation. All policy comparisons use complete simulator episodes on identical seeds and scenario variants.

The selected runtime does not output acceptance probabilities. This is intentional. A directional probability for A accepting B, the reverse probability, and a joint mutual-acceptance probability are different targets. The available evidence did not support a calibrated pair-level estimator that survived holdout testing.

4. Method

The daily process is:

```
observable ask-state
↓
clarify missing hard constraints within budget
↓
refresh observable match-state
↓
build the reciprocal feasible graph
↓
score only feasible edges
↓
build greedy and guarded global batches
↓
```

```
return one deterministic JSON batch
```

4.1 Clarification policy

The daily ask budget is 12. A hard-constraint bundle costs 3, so the policy can ask at most four members per day.

The policy asks an available member when:

- at least one hard constraint is unknown; and
- the member has not declined the hard-constraint bundle.

A declined answer stays unknown. The policy does not spend more budget asking for the same declined bundle. After clarification, the evaluator refreshes the state before the match phase.

The current rule takes the first eligible members in observable state order. This is deterministic and safe, but it is not graph-aware. It can spend budget on a member with few plausible partners.

We tested a graph-based expected-unlock rule. It won its 30-episode screen, 0.3833 to 0.2500, but lost a fresh 60-episode confirmation, 0.2583 to 0.3083. We rejected it rather than promote a tuning win.

4.2 Reciprocal feasible graph

After clarification, the policy keeps only available members with known hard constraints. For every possible pair, it calls the official reciprocal eligibility function.

An edge exists only when both people satisfy all of the following conditions:

- each person's accepted age range;
- each person's stated genders to meet;
- matching relationship structure;
- smoking and partner-smoking rules;
- children-related rules;
- mutually acceptable zones; and
- at least one shared schedule window.

The policy also removes pairs introduced earlier. Batch construction prevents one member from appearing in two pairs.

4.3 Observed compatibility

For a feasible pair (u, v) , the policy checks seven soft fields:

- relationship goal;
- relationship pace;
- lifestyle;
- conversation preference;
- emotional availability;
- space for a relationship; and
- relocation preference.

Let K be this field set. The compatibility count is:

$$C(u, v) = \sum_{k \in K} 1[\text{field } k \text{ is observed for both members and the values are equal}]$$

A missing value contributes zero. It is not treated as agreement or disagreement. This is conservative, but it favors pairs with more jointly observed fields. Across the ten public day-30 pools, only 36% to 46% of soft fields are observed. We therefore treat $C(u, v)$ as a ranking rule, not as a calibrated compatibility probability.

The seven fields have equal weight. Equal weighting is a transparency choice, not a claim that every field predicts MSMI equally. Goal-only, hand-weighted, and learned alternatives did not produce a repeatable holdout gain.

4.4 Mature history signal

Before day 20, edge score is:

$$S(u, v) = 100 * C(u, v)$$

From day 20, the policy computes two Bayesian-smoothed member rates from already observed introduction responses:

$$\begin{aligned} \text{response_rate}(u) &= (\text{responses}(u) + 4 * 0.765) / (\text{response_trials}(u) + 4) \\ \text{accept_rate}(u) &= (\text{yes_responses}(u) + 4 * 0.46) / (\text{recorded_responses}(u) + 4) \end{aligned}$$

The member history value is:

$$H(u) = \text{clip}(\text{logit}(\text{response_rate}(u)) + \text{logit}(\text{accept_rate}(u)), -20, 20)$$

The edge score becomes:

$$S(u, v) = 100 * C(u, v) + H(u) + H(v)$$

The response and acceptance prior means are 0.765 and 0.46. Later training rollouts measured 0.7592 and 0.4574 across 18,149 introductions, which is close to those development choices. Prior strength four is a hand-chosen small-sample regularizer. It was not fitted by the rejected final outcome model.

The bound is important. The pair history term lies between -40 and 40, so its full range is 80. One compatibility point is worth 100. History therefore cannot reverse a one-point compatibility advantage on an individual edge.

The policy uses response and Yes history only. It does not use a member's second-meeting history in the runtime score because that label is rarer and is observed only after several selected events in the outcome funnel.

4.5 Daily allocation

The policy builds two candidate batches.

The greedy batch sorts edges by descending score, then by pair ID for stable ties. It accepts the next edge only when neither member is already used.

The global batch starts with Edmonds' blossom algorithm to find a maximum-cardinality matching in the general graph. A deterministic local search then improves total policy score without reducing pair count. The local search uses one-edge replacements and two-edge exchanges for at most 20 rounds.

The global batch replaces the greedy batch only when both tests pass:

$$\begin{aligned} \text{number of global pairs} &> \text{number of greedy pairs} \\ \text{total score of global batch} &\geq \text{total score of greedy batch} \end{aligned}$$

This is the safe-cardinality guard. It prevents global matching from changing a batch only because a different maximum matching exists.

The refinement does not prove maximum total weight among every maximum-cardinality matching. Its purpose is narrower: find a larger batch, improve it locally, and use it only when the observable guard passes.

4.6 Waiting policy

The policy does not impose a minimum score threshold. It matches feasible edges rather than waiting for a speculative future arrival that the policy cannot observe.

A member waits when:

- no reciprocally feasible candidate exists;
- a hard constraint remains unknown or declined;
- the member is unavailable;

- all feasible candidates are used by better-ranked edges; or
- the pair was introduced earlier.

We tested explicit waiting thresholds. They reduced opportunities and did not produce a stable MSMI gain, so they are not in the selected runtime.

5. Why the policy preserves reciprocal constraints

The safety argument has four steps:

1. The policy creates edges only from members with known hard fields.
2. The official `eligibility` function must return `feasible` for each edge.
3. Both batch builders return a matching, so one member appears at most once.
4. The simulator validates the complete batch atomically before applying it.

The score operates only after Step 2. No history value, compatibility count, or allocation gain can create an edge that failed reciprocal feasibility.

The implementation tests empty states, declined answers, deterministic output, unknown hard fields, repeated pairs, overlapping batches, memory limits, and maximum-cardinality correctness on small graphs.

6. Missing, declined, and delayed information

Information state	Policy treatment	Reason
Missing hard field	Block the edge and ask when budget permits	Unknown is not feasible
Declined hard field	Keep unknown and do not ask again	Refusal must remain respected
Missing soft field	Add no compatibility point	Missing is not agreement or disagreement
Pending response	Do not create a response trial yet	The response deadline has not passed
Observed null response	Count a response trial but no response	Missing response is not a No preference label
Recorded Yes or No	Update response and acceptance history	The label is observable
Pending date or second intention	Do not infer failure	The observation window is incomplete
New episode	Reset memory to null	Private episodes must remain independent

This treatment avoids future leakage and prevents right-censored outcomes from becoming false negative labels.

7. Experimental design

7.1 Public setting

The supplied dataset contains 2,000 synthetic adults in ten disjoint pools, 613 historical introductions, 1,270 observable feedback events, and 1,129 template conversation records. We use the published train, validation, and development-test pool split. We do not treat day-30 snapshots as outcomes from our earlier policy decisions.

Each evaluation episode has 60 decision days and 40 follow-up days. We test all six public scenario families:

- development;
- sparse geography;
- cold start;
- delayed dates;

- shifted outcome weights; and
- changing response conditions, called drift.

7.2 Fair comparison rules

Every direct comparison uses identical seeds and scenario variants for both methods. Early screens and final holdouts use disjoint seed blocks. We judge MSMI first, then use the official tie-break order.

We do not select a policy from its best seed. A small screen can nominate a candidate, but a fresh confirmation decides whether the candidate remains credible. Rejected methods remain in the decision log and research code, not in the submission runtime.

7.3 Main seed registry

Purpose	Seeds	Variants	Episodes per method
Public smoke comparison	101-103	All six	18
Safe-cardinality validation	1101-1110	All six	60
Day-20 history screen	3901-3910	All six	60
Day-20 history final holdout	4001-4020	All six	120
Allocation activation audit	5301-5303	All six	18
Missing-field credit screen	5401-5405	All six	30
Missing-field credit confirmation	5501-5510	All six	60

The rejected learned scorer used seeds 2001-2040 across all six variants to create 240 training episodes and 18,149 assignment-time feature rows. Its tuning and holdout seeds were separate from this training block.

7.4 Uncertainty analysis

For the final history comparison, we compute an episode-level paired difference on 120 matched episodes. A deterministic paired percentile bootstrap resamples within each scenario family. We use 20,000 resamples, bootstrap seed 1701, and a 95% interval.

The interval measures sampling uncertainty in these public simulator worlds. It does not cover every form of uncertainty, such as private-world shift or a mismatch between the simulator and real people.

8. Baselines and ablations

The three official baselines are:

- **greedy**: clarify hard constraints, then greedily rank feasible pairs by observed soft agreement;
- **no_asks**: never clarify missing fields; and
- **random**: choose random feasible pairs.

The selected policy also provides focused ablations:

Mode	Question tested
adaptive_legacy	Does day-20 history help?
adaptive_greedy	Does global allocation help?
adaptive_always_max	Is the safe-cardinality guard necessary?

The supplied seed-101 references are interface smoke tests, not reliable performance estimates:

Method	Episodes	MSMI score	Coverage	Mutual acceptances per 100	Ask cost
Greedy	6	0.500	0.408	7.583	205
No clarification	6	0.333	0.174	2.833	0
Random feasible	6	0.250	0.412	6.667	205

The full Round 2 comparison will run all four policies on identical seeds and all six scenario families. Round 1 method selection uses the larger matched blocks below rather than the six-episode references.

9. Results

9.1 Safe-cardinality allocation improved the selected point estimate

On seeds 1101-1110 across all six variants, safe-cardinality scored 0.5417, compared with 0.5250 for greedy. Coverage was 0.3355 versus 0.3348.

Method	Episodes	MSMI score	Coverage	Mutual acceptances per 100	Ask cost
Safe-cardinality	60	0.5417	0.3355	5.5667	195.75
Greedy	60	0.5250	0.3348	5.7500	195.75

The primary difference is small. The result supports the guard as a useful allocator, not as a guarantee of private-evaluation improvement.

9.2 The global branch is uncommon but active

Allocation telemetry covered 18 episodes and 1,080 decision days:

Audit measure	Count
Decision days	1,080
Days with at least one feasible edge	584
Days where maximum cardinality exceeded greedy	52
Days where the global batch also passed the score guard	26

The policy used the global batch on 2.41% of all days and 4.45% of nonempty days. It did not activate in the sparse scenario during this audit. Sparse geography usually leaves too few connected alternatives for global matching to improve cardinality.

9.3 Day-20 history won two larger blocks but lost the small public block

Evaluation block	Episodes per method	Guarded history	No-history ablation	Difference
Screen, seeds 3901-3910	60	0.5000	0.4750	+0.0250
Final holdout, seeds 4001-4020	120	0.2708	0.2500	+0.0208
Public check, seeds 101-103	18	0.3611	0.3889	-0.0278

The final holdout result by scenario was:

Scenario	Guarded history	No history	Difference	Guarded coverage	Guarded mutual acceptances per 100
Development	0.275	0.225	+0.050	0.378	7.100
Sparse	0.050	0.050	0.000	0.158	1.125

Scenario	Guarded history	No history	Difference	Guarded coverage	Guarded mutual acceptances per 100
Cold start	0.325	0.325	0.000	0.333	5.325
Delayed	0.375	0.375	0.000	0.383	6.400
Shift	0.350	0.325	+0.025	0.385	6.100
Drift	0.250	0.200	+0.050	0.383	6.875
Equal-weight mean	0.2708	0.2500	+0.0208	0.3367	5.4875

No scenario mean fell on this holdout. Guarded history produced 65 MSMI outcomes, compared with 60 for the no-history ablation. Coverage rose from 0.3353 to 0.3367, and mutual acceptances rose from 5.3500 to 5.4875 per 100 arrived members. Mean clarification cost was unchanged at 201.45.

The paired bootstrap interval was $[-0.0167, 0.0583]$. Because the interval includes zero, this experiment does not prove a reliable improvement. We choose guarded history as the score-seeking default because it won the larger screen and holdout, added five outcomes, and had no scenario-level mean loss. We retain `adaptive_legacy` as the conservative fallback.

9.4 Outcome funnel and waiting behavior

The 120-episode final holdout gives the following mean episode funnel:

Stage	Guarded history	No history
Assignments	76.38	76.55
Mutual acceptances	10.98	10.70
Dates	8.77	8.62
MSMI outcomes, total across block	65	60
Missing feedback	41.11	41.11
Distinct served members	67.34	67.06
Unserved members	132.66	132.94

For guarded history, the median first-introduction wait among served members was four days. The 90th percentile was 17 days, and the maximum was 58 days. The no-history ablation had the same median and 90th percentile. These values show that the final outcome is rare even after mutual acceptance, and that supply constraints leave many members unserved.

9.5 Negative results shaped the final policy

Candidate	Screen result	Fresh result	Decision
Goal-only score	0.5250 vs 0.5167	0.3375 vs 0.3583	Reject
Expected-unlock clarification	0.3833 vs 0.2500	0.2583 vs 0.3083	Reject
Missing-soft credit 0.25	0.5333 vs 0.5000	0.4500 vs 0.5000	Reject
Learned history scorer	0.4667 vs 0.3500	0.4167 vs 0.5083	Reject
Conservative pair formula	0.3250 vs 0.3083	0.3792 vs 0.4208	Reject

These reversals are the central research lesson. Rare outcomes make small screens unstable. Complexity was easy to add and hard to validate. The final policy keeps only components that remained safe and produced the strongest repeatable point estimates.

10. Selection rule

A challenger must satisfy all of the following conditions before replacing the incumbent:

1. It improves the primary score on a matched-seed screen.
2. It repeats the gain on a fresh seed block.
3. It preserves episode validity and reciprocal constraints.
4. It stays deterministic, offline, and within resource limits.
5. It does not hide a primary-metric loss behind a tie-break gain.

We also report a paired 95% interval. An interval above zero supports a strong improvement claim. An interval that includes zero means the measured gain is uncertain, even when the point estimate wins.

The confidence gate marks the selected history gain as uncertain. This does not erase the point estimate, but it limits the claim we make from it.

11. Where the method may fail

Failure case	Why it matters	Current response
Sparse geography	Few feasible edges exist, so allocation has little freedom	Preserve feasibility and accept lower coverage
Declined hard constraints	Some members can never become safely matchable	Respect the decline and do not infer an answer
Cold start	New members have no behavior history	Use compatibility alone before history exists
Delayed feedback	Recent assignments have unresolved outcomes	Use only feedback already observed
Uneven history	Active members can have more trials than new members	Apply Bayesian smoothing and a hard history bound
Missing soft fields	Raw equality favors more complete profiles	Give missing values zero and report the bias
State-order clarification	Budget may go to a low-value member	Keep the tested simple rule; graph-aware asks failed confirmation
Competition for one candidate	Greedy selection can block more total pairs	Compare with guarded maximum cardinality
Rare MSML outcomes	One outcome moves a small experiment sharply	Use matched seeds, fresh holdouts, and uncertainty intervals
Scenario shift	Public relationships may not hold in private worlds	Test all six variants and avoid unconfirmed fitted models
Pair-specific preference	General history may not predict one particular pair	Keep history secondary to observed pair compatibility

The local quality refinement is also limited. It does not solve exact maximum weight among all maximum-cardinality matchings. More introductions can improve coverage without improving MSML.

12. Runtime, packaging, and reproducibility

The submitted runtime uses Python 3.10 or later and the Python standard library. It has no external model, API key, network dependency, GPU requirement, or inference randomness. Deterministic sorting resolves ties.

At 200 members, graph construction checks at most 19,900 unordered pairs. The general matching step is cubic in the number of graph vertices. It remained within the competition limit in the tested episodes.

The seed-101 development benchmark made 120 real policy-process calls:

Check	Measured	Limit
Mean call latency	0.136 s	10 s
95th percentile latency	0.166 s	10 s
Maximum call latency	0.183 s	10 s
Largest request	474,833 bytes	1,048,576 bytes
Largest response	911 bytes	1,048,576 bytes

The offline container passed the official isolated seed-101 development check. It was valid and eligible, scored 0.5, reached 0.465 coverage, and produced 10.0 mutual acceptances per 100. The last verified image size was about 45.2 MB, below the 2 GiB limit.

Run the required checks from the repository root:

```
python -m unittest -v
python verify_data.py
docker build -t sequential-policy:submission .
python evaluate.py --image sequential-policy:submission --seeds 101 --variants development
--output results/container_check.json
```

Run the final history confidence audit with:

```
python experiments/confidence_audit.py --incumbent adaptive_legacy --challenger adaptive --seeds
4001,4002,4003,4004,4005,4006,4007,4008,4009,4010,4011,4012,4013,4014,4015,4016,4017,4018,4019,4020
--variants all --resamples 20000 --output results/history_confidence_4001_4020.json
```

Run the allocation activation audit with:

```
python experiments/allocation_telemetry.py --seeds 5301,5302,5303 --variants all --output
results/allocation_telemetry_5301_5303.json
```

The repository currently passes 61 tests. `verify_data.py` verifies all 2,000 profiles and 71 published files.

13. Data, model, and dependency provenance

- All member records and outcomes are synthetic and come from the supplied version 1.0.0 generator and public data files.
- The selected policy uses no external dataset or pretrained model.
- The selected runtime has no fitted inference asset. The rejected outcome model remains available only as reproducible research evidence.
- The rejected model used assignment-time observable features and declared public synthetic seeds 2001-2040. It does not run in the container.
- The runtime depends only on Python's standard library and the committed repository files.
- General development tools helped edit, test, and review the repository. They do not run during evaluation and do not provide policy features or scores.

14. Limits of interpretation and later product use

The profiles, preferences, and outcomes are invented. The simulator score does not establish real relationship quality, fairness, safety, or product impact. The policy should not be deployed on real people from this evidence alone.

Later authorised evaluation would need:

- a reviewed adapter from production records to the documented policy schema;
- current reciprocal constraints checked at decision time;
- consent, privacy, retention, and access controls;

- outcome definitions that reflect the real service;
- subgroup and missingness analysis;
- calibration and prospective validation on authorised data; and
- human review, appeal, and monitoring processes.

The public synthetic identifiers are opaque. We make no claim that the current fields are sufficient for real matching or that equal soft-field agreement is fair across real groups.

15. Final claim

Guarded-history safe-cardinality is our strongest rules-compliant submission after matched-seed testing. Its main strengths are clear boundaries:

- reciprocal feasibility is enforced before scoring;
- observed compatibility remains the primary edge signal;
- delayed history starts only on day 20 and cannot override one compatibility point;
- global matching is accepted only when it adds pairs without lowering total policy score; and
- candidates that win only a small screen stay out of the runtime.

We do not claim a guaranteed win or a reliable score of 0.9. The final history holdout favors the selected policy by 0.0208 MSMI per 100, but its 95% interval includes zero. The defensible conclusion is narrower: this policy is safe, reproducible, competitive on the tested simulator blocks, and honest about what the evidence has not established.

Evidence map

- Runtime policy: [adaptive.py](#)
- JSON entry point: [policy.py](#)
- Algorithm explanation: [docs/SAFE_CARDINALITY_ALGORITHM.md](#)
- Confidence method: [docs/CONFIDENCE_AUDIT.md](#)
- Experiment decisions: [docs/DECISIONS.md](#)
- Learned-model rejection: [docs/OUTCOME_MODEL_EXPERIMENT.md](#)
- Pair-scorer rejection: [docs/PAIR_SCORER_EXPERIMENT.md](#)
- Score search: [docs/SCORE_TARGET_ANALYSIS.md](#)
- Public interface: [docs/POLICY_INTERFACE.md](#)
- Public data contract: [docs/DATA_CONTRACT.md](#)